



Scaling The Data Layer In Distributed Systems

Technology

📅 December 26, 2025 👤 Admin 💬 Leave A Comment

Scaling a stateless system is relatively straightforward. We can horizontally scale application servers by adding more instances behind a load balancer, and the system continues to work as expected.

But what happens to the database?

How can a database running on a single machine handle requests coming from N horizontally scaled application servers? Databases must scale too — and this is where things get interesting.

The First Idea: Replication

A natural first thought is:

“Let’s just replicate the database and create multiple copies.”

At first glance, this sounds reasonable. But the moment we try to scale traditional SQL databases using replication, we run into fundamental constraints.

Why SQL Databases Don’t Scale Easily

SQL databases are built around powerful guarantees:

Joins across tables

Cross-row and multi-table transactions

Global constraints (foreign keys, unique keys, auto-increment IDs)

All of these assume one critical thing:

All data lives in one logical place.

Once we attempt to shard or distribute SQL data:

Joins turn into cross-network calls

Transactions span machines

Search

Search

RECENT POSTS

- » [What is Microsoft Planner and who is it for](#)
- » [You need to know what the tilde \(~\) does in Excel](#)
- » [To harness crypto ingenuity, financial threat must first be neutralised](#)
- » [Ethereum-Solidity Quiz Q5: What is the Private Mempool?](#)
- » [HOW TO CREATE AN ELASTIC BEANSTALK ON AWS](#)

RECENT COMMENTS

💬 [admin](#) on [What is Microsoft Planner and who is it for](#)

ARCHIVES

- 📅 [December 2025](#)
- 📅 [November 2025](#)
- 📅 [October 2025](#)
- 📅 [July 2025](#)
- 📅 [June 2025](#)

Locks stop working globally

As a result:

Complexity explodes

Performance degrades

Application logic becomes significantly more complex

This doesn't mean SQL cannot be sharded — it can — but doing so safely requires heavy coordination and careful design.

The Leader-Follower Model

To improve scalability, many systems move to a leader-follower replication model.

Press enter or click to view image in full size

How It Works

A single leader (primary) accepts all writes

Followers (replicas) copy data from the leader

Reads may be served from followers to scale read traffic

This gives us:



One source of truth for writes

Multiple copies for read scaling and fault tolerance

But a key question remains:

May 2025

April 2025

March 2025

January 2025

December 2024

October 2024

September 2024

August 2024

June 2024

May 2024

April 2024

March 2024

February 2024

January 2024

December 2023

November 2023

June 2023

May 2023

April 2023

December 2022

October 2022

July 2022

June 2022

May 2022

March 2022

December 2021

April 2020

September 2019

August 2019

May 2019

How does the leader propagate writes to followers?

The answer lies in replication strategies.

Replication Strategies in Leader–Follower Systems

The leader writes data locally

The leader waits for acknowledgements from a quorum (majority) of replicas

Only then does it respond to the client

This ensures:

Strong consistency

No stale reads (when reads also use quorum or leader)

Tradeoffs:

Higher write latency

Reduced availability if replicas are slow or unavailable

Importantly, strong consistency does not require waiting for all followers — waiting for a majority is sufficient.

The leader writes data locally

It sends replication messages to followers

It responds to the client without waiting for most acknowledgements

This model:

Improves write latency and throughput

Allows reads from followers to return stale data temporarily

Relies on the system eventually converging

This is a deliberate tradeoff:

Availability and performance over immediate consistency

The Leader Bottleneck Problem

Even with replication, a fundamental limitation remains:

All writes still go through a single leader.

As write QPS grows (e.g., 10K+ writes per second):

The leader becomes a bottleneck

CPU, I/O, and replication overhead increase

Vertical scaling eventually hits limits

At this point, simply adding more replicas doesn't help — because replicas don't accept writes.

This is where partitioning (sharding) becomes necessary.

📅 February 2019

📅 October 2018

📅 September 2018

📅 December 2017

CATEGORIES

📁 business

📁 entertainment

📁 general

📁 health

📁 politics

📁 science

📁 sports

📁 technology

The Entry of NoSQL

NoSQL databases emerged to solve exactly this problem:

scaling writes and reads horizontally at massive scale.

Key motivations behind NoSQL systems:

High write throughput

High availability

Fault tolerance

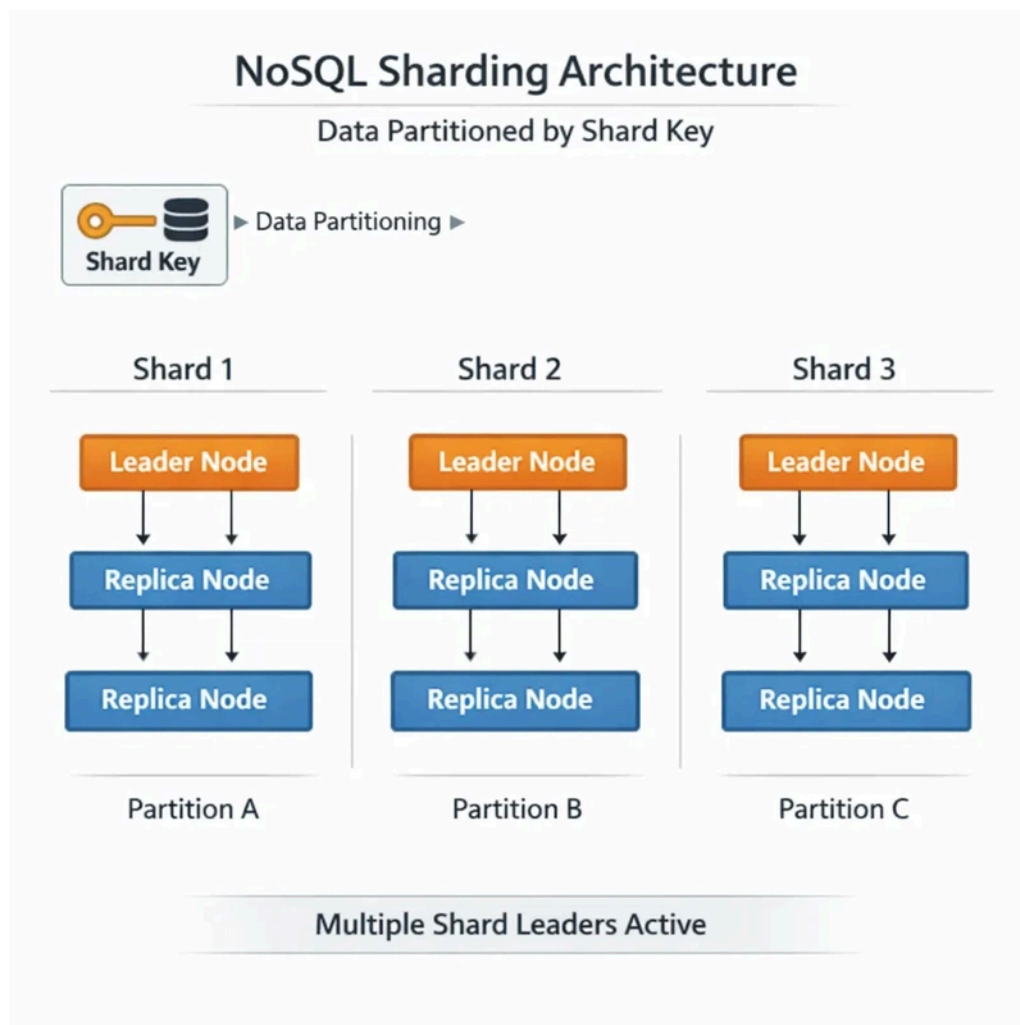
Geo-distribution

Low latency at scale

Rather than optimizing purely for normalization and storage efficiency, NoSQL systems are designed to scale by default.

Sharding in NoSQL Databases

NoSQL databases distribute data using sharding.



How Sharding Works

A partition (shard) key is chosen (e.g., `user_id`)

The key is hashed or range-mapped

Data is distributed across shards

Each shard owns a mutually exclusive subset of data

Common partitioning strategies:

Consistent hashing

Range-based partitioning (A–G, H–M, etc.)

Choosing the correct shard key is critical:

A poor shard key can cause hotspots and destroy scalability.

Replication Inside a Shard

Each shard is still replicated for fault tolerance.

This is where two NoSQL replication models appear.

Model 1: Leader–Follower per Shard

Used by systems like:

MongoDB

DynamoDB

HBase

Per Shard:

One leader

Multiple followers

Write Flow:

Client routes request to shard leader

Leader writes data

Leader propagates to followers

Leader waits for:

Quorum acknowledgements (stronger consistency), or

Minimal acknowledgements (eventual consistency)

Read Flow:

Reads may go to leader (fresh)

Or followers (faster, possibly stale)

This model is simple and efficient but still has:

Leader hotspots

Leader failover complexity

Model 2: Quorum-Based (Leaderless) Replication

Used by systems like:

Cassandra

Riak

Dynamo-style designs

Per Shard:

No leader

All replicas are equal

Write Flow:

Client sends write to any replica (coordinator)

Coordinator sends write to all replicas

Write succeeds when W replicas acknowledge

Read Flow:

Client reads from R replicas

Latest version wins

Inconsistencies are repaired in the background

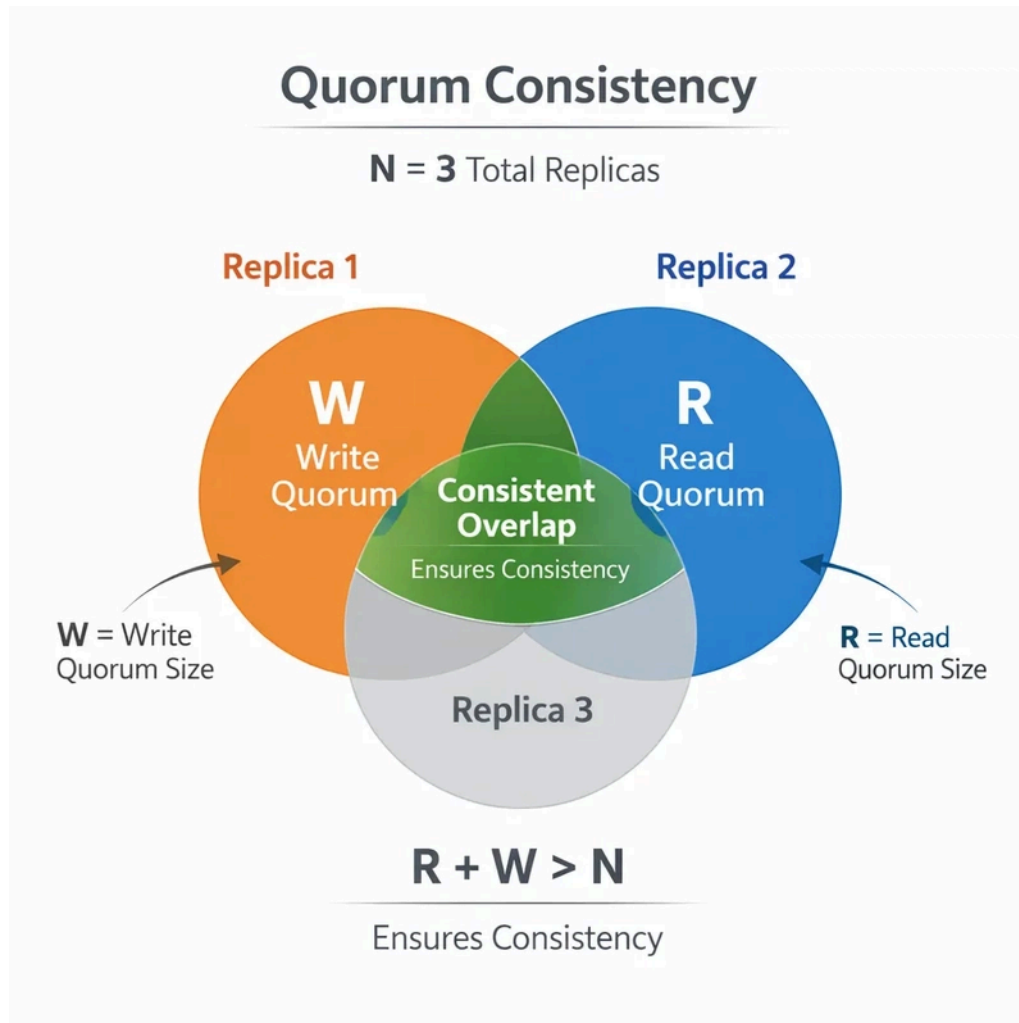
This model:

Avoids leader bottlenecks

Improves availability

Requires conflict resolution and versioning

Quorum Explained (Simply)



Let:

N = total replicas per shard

W = replicas required to acknowledge a write

R = replicas required for a read

Rules:

Strong consistency: $R + W > N$

Eventual consistency: $R + W \leq N$

Examples:

Banking, inventory → strong consistency

Social feeds, likes, analytics → eventual consistency

Quorum decisions are made per shard, not globally.

Final Takeaway

SQL databases struggle with horizontal write scaling due to strong global guarantees

Leader–follower replication improves read scaling but not write scaling

NoSQL databases solve this by sharding data ownership

Each shard scales independently

Replication within shards is handled using leader-based or quorum-based models

Consistency is a configurable tradeoff, not a fixed property

Modern data systems don't ask "Can we scale?" — they ask "What consistency are we willing to trade for scale?"



Templates let you quickly answer FAQs or store snippets for re-use.

Are you sure you want to hide this comment? It will become hidden in your post, but will still be visible via the comment's permalink .

Hide child comments as well

Confirm

For further actions, you may consider blocking this person and/or reporting abuse

👁 Tagged backend coding community Database development engineering inclusive java software systemdesign

◀ Unlock Your...

HOW TO CRE...

RELATED POSTS

Amazon FSx For NetApp
ONTAP Now Integrates
With Amazon S3 For
Seamless Data Access

December 2, 2025

admin

The Latest AI News We
Announced In November

December 5, 2025

admin

Ethereum-Solidity Quiz
Q5: What Is The Private
Mempool?

December 26, 2025

admin

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *

Email *

Website

☐

Save my name,
email, and website
in this browser for
the next time I
comment.

Post Comment